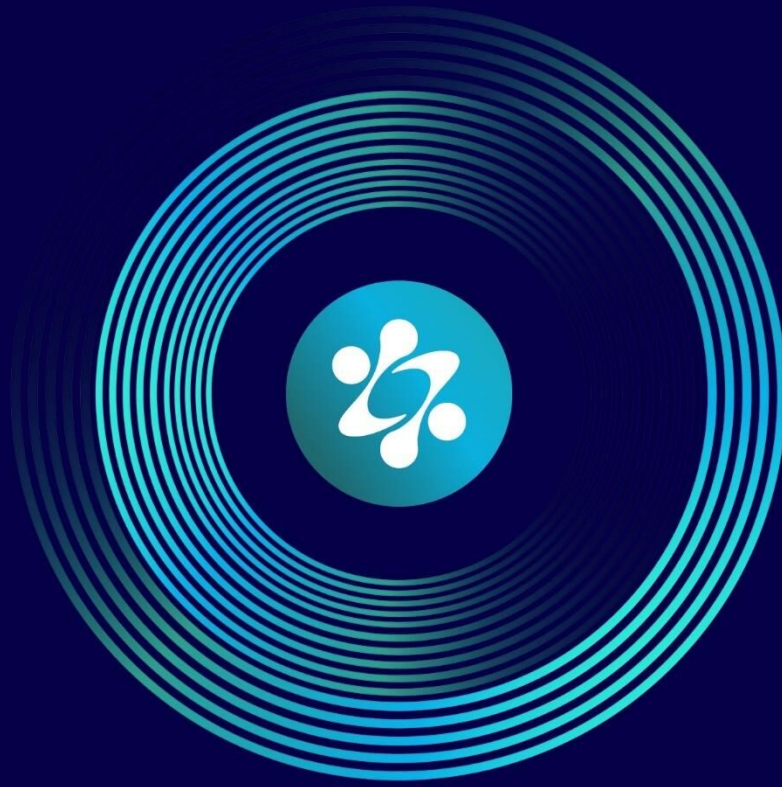




Unmasking: Technological Advancement and Evolution of MuddyWater in 2024

February 2025

TLP: CLEAR



INCD
Israel National
Cyber Directorate

Table of Contents

Executive Summary.....	4
Introduction	5
Phishing Activity – Worldwide and in Israel	9
Technological Advancement and Expansion	14
Tradecraft.....	16
Bugsleep (PE64,C++).....	16
Blackout (PE64, C++)	17
AnchorRat.....	22
CannonRat.....	25
Neshta	27
SOCKS5	30
C2 Frameworks	33
Sad C2.....	33
SAD C2 Tools	36
HAVOC Framework.....	42
Summary of Findings and Investigation Report.....	44

Introduction

Executive Summary

This report provides an in-depth analysis of the activities of the Iranian cyber-attack group MuddyWater within the Israeli cyber domain, focusing on their tactics, techniques, and procedures (TTPs), tools, and primary attack targets.

The key topics covered in the report include:

- **Tactics, Techniques, and Procedures (TTPs):** A review of techniques used by the group, including targeted phishing, spoofing, persistence methods such as COM Hijacking, and the exploitation of communication protocols such as HTTP and DNS.
- **Campaigns:** An overview of the group's modus operandi, with a focus on the planning and execution phases of their attack operations based on specific objectives.
- **Tools:** An analysis of the tools employed by the group, including the use of legitimate tools (LOTL) for malicious purposes (dual-use tools) and the development of custom-made tools, as well as the exploitation of protocols such as SOCKS5.
- **Infrastructures:** Insights into the command and control (C2) frameworks used by the group, including Havoc C2 and Sad C2. The report also discusses the use of legitimate platforms (LOTS), such as cloud storage services and collaboration tools, which the group leverages to obscure its activities and distribute malicious files.

The report emphasizes the flexibility and innovation of the MuddyWater attack group, which uniquely combines existing tools with custom solutions. Understanding the group's operational methods is crucial for developing effective defenses against this threat.

Introduction

MuddyWater¹ is an Iranian threat group operating under the Iranian Ministry of Intelligence and Security (MOIS)² since 2017. The group is also known by other names in the cyber domain, such as Static Kitten, Zagros, and Mango Sandstorm. The group primarily focuses on the Middle East, alongside operations in other countries, including Israel, Turkey, Afghanistan, Pakistan, the United Arab Emirates, Iraq, the United Kingdom, Azerbaijan, the United States, Egypt, and Nigeria. This wide geographic distribution highlights the group's global impact and its capability to operate across complex arenas. Furthermore, its international scope underscores the significant resources at its disposal and its ability to execute sophisticated campaigns in diverse and dynamic operational environments.

In the aftermath of the outbreak of the "Iron Swords" War, a marked increase in cyber activity attributed to the MuddyWater group was observed within Israel. This resurgence follows a period of relative inactivity, which may have been influenced by the public exposure of the group's members at the CyberTech Tel Aviv 2023 conference. The exposure, which was incorporated into a speech delivered by the Head of the Israel National Cyber Directorate (INCD), heightened awareness of the group's operations and their potential implications.

¹ MuddyWater

<https://attack.mitre.org/groups/G0069/>

² The Ministry of Intelligence and Security of Iran (MOIS), also known as Vezarat-e Ettela'at Jomhuri-ye Islami-ye Iran, is the central intelligence agency of Iran. The ministry was established after the Islamic Revolution in 1979, and its responsibilities include gathering domestic and international intelligence, conducting counter-intelligence operations, and ensuring Iran's national security.

The ministry plays a central role in monitoring political activities, preventing espionage, and addressing perceived threats to the regime. In addition, it is involved in cyber activities, surveillance, and foreign intelligence operations. The Ministry of Intelligence operates under the supervision of the Supreme Leader of Iran to ensure alignment with the country's ideological and strategic objectives.

This development highlights the group's strategic adaptability in responding to changes in the geopolitical environment, as well as its ability to leverage such shifts to advance its objectives. This behavior reflects a proactive and flexible operational approach. Within Israel, the group has demonstrated a targeted focus on sectors vital to the functionality of both the state and the economy, such as local authorities, civil aviation, tourism, healthcare, telecommunications, information technology, and small and medium-sized enterprises (SMEs). The group's overarching objective, as indicated by its global activities—especially in Israel—appears to be intelligence gathering (Computer Network Exploitation, CNE).

A comprehensive analysis of the group's operations reveals a strategic emphasis on establishing sustained access within targeted organizations to gather long-term, strategic intelligence. This focus is particularly evident in its repeated attempts to infiltrate telecommunications companies.

The group employs diverse social engineering tactics, such as spear-phishing, alongside exploiting known system vulnerabilities—particularly in VPN infrastructure (T1190³)—to infiltrate organizational networks. After gaining access, the group relies on a variety of malicious tools, as well as the abuse of legitimate tools (LOTL), to establish persistent footholds and exfiltrate sensitive data from organizational environments.

Historically, the group primarily relied on publicly available malware, which it adapted to suit its objectives. However, over the past year, there has been a

³ Exploit Public-Facing Application T1190
<https://attack.mitre.org/techniques/T1190/>

discernible shift towards the independent development of custom-tailored tools. A comprehensive overview of these tools is provided later in this report.

Furthermore, the group frequently exploits legitimate platforms (LOTS), including cloud storage services, collaboration tools, and content delivery networks, to conceal its activities and facilitate the distribution of malicious files.

Phishing Activity

Phishing Activity – Worldwide and in Israel

The threat group employs a range of phishing campaigns, both broad-based and targeted (spear-phishing), across multiple countries to infiltrate the organizational networks of its victims. The identified attack methods include:

1. **Conventional Spear-Phishing:** Creating accounts on free email platforms to send malicious links or attachments to carefully selected targets.
2. **Compromised Email Accounts:** Gaining control of legitimate email accounts and leveraging active email threads to distribute malicious content, thereby making the emails appear as legitimate communications from the compromised account holder.
3. **Spoofing:** Sending phishing emails from addresses designed to resemble those of reputable organizations, in an effort to deceive recipients into engaging with malicious content.

To enhance the credibility of their phishing campaigns, the group utilizes advanced techniques, including:

1. **Impersonating Recognizable Entities:** Constructing fake identities based on trusted organizations to enhance the perceived legitimacy of the communication.
2. **Distributing Emails from Compromised Organizations:** Leveraging breached organizational email accounts to send phishing messages, utilizing content relevant to the organization's activities to increase the likelihood of success.
3. **Tailored Messaging:** Moving beyond generic phishing templates, the group crafts personalized messages tailored to the victim's specific area of operation.

4. **Exploiting Internal Organizational Infrastructure:** Utilizing internal resources and infrastructure within compromised organizations to increase the authenticity and effectiveness of the campaigns.

These tactics underscore the group's reliance on sophisticated social engineering strategies, designed to deceive victims in a targeted and customized manner in order to gain access to sensitive resources.

In 2024, the group initiated large-scale spear-phishing campaigns across the Middle East, with a particular emphasis on targets within Israel. These attack vectors commonly involved links to reputable file-sharing services, such as Egnyte and OneDrive, which were used to distribute compressed files. These files often contained remote management tools (RMM), enabling attackers to establish remote access to compromised systems. While these RMM tools are legitimate and widely employed for professional purposes, they were exploited by the attackers for malicious activities.

The use of RMM tools provides the attackers with full remote control over the compromised devices, including the ability to transfer files, control input devices such as the mouse and keyboard, monitor screens, and more. This level of access allows the attackers to maintain a persistent presence within the organizational network, facilitating subsequent actions such as deepening network infiltration, collecting data, and preparing for further attacks.

For example, during one attack, MuddyWater used RMM to remotely exfiltrate sensitive documents while simultaneously monitoring the victim's screen activity to gather further information about the organization. This level of control allowed

the attackers to perform customized follow-up actions, ensuring their foothold in the organizational network and maintaining long-term access.

The group also occasionally conducts broad phishing campaigns that are not specifically targeted at a particular nation or sector. These campaigns typically involve impersonating legitimate companies to deceive victims into disclosing sensitive information.

In the most recent broad campaign identified, the INCD detected over 10,000 email accounts, including several Israeli accounts, that received an email impersonating a legitimate software update from "Microsoft." In actuality, the email contained a link designed to download and install a remote management tool (RMM).

During the attack, the attacker utilized a dedicated API key embedded within the email content, combined with the execution of PowerShell code. This granted the attacker direct access to the agent after its installation on the endpoint. This access enabled the attacker to control the compromised machine and continue executing malicious activities remotely ⁴.

Recently, a new trend has been identified in which the threat group distributed malicious tools as part of its phishing campaigns, alongside campaigns delivering RMM tools. Some of these tools are modified versions of publicly available tools, particularly those found on the GitHub platform. However, there is a clear tendency for most tools to be independently developed by the threat group.

⁴ https://www.gov.il/he/pages/alert_1826b

This trend began around May 2024, with various phishing campaigns worldwide distributing customized versions of a backdoor, known within the research community as BugSleep⁵. The remainder of this report provides an in-depth review of this tool, its functionality, and its unique characteristics.

⁵ <https://research.checkpoint.com/2024/new-bugsleep-backdoor-deployed-in-recent-muddywater-campaigns/>

Technological Advancement and Expansion



INCD
Israel National
Cyber Directorate

Technological Advancement and Expansion

An analysis of MuddyWater's activities over the past year reveals a significant enhancement in its capabilities, reflected in its constant adaptation to the evolving threat landscape and adoption of new technologies. These developments are evident in the following:

1. **In-House Development of Tools and Frameworks:** The group demonstrates impressive adaptability by developing its own tools and frameworks. These innovations enable the group to overcome security measures and tailor its operations to various attack tactics, such as lateral movement within organizational networks and data exfiltration. A detailed review of these tools is provided later in the report.
2. **Utilization of DLL Side-Loading (T1574/0023⁶):** This technique relies on the loading and execution of malicious DLLs by exploiting the system's loading mechanisms. While it shares similarities with the DLL Search Order Hijacking technique (T1574/0014⁷), it differs in its approach. Instead of passively planting malicious DLLs and waiting for them to be loaded by a user application via the DLL Search Order, DLL Side-Loading employs a proactive strategy.

In this approach, the attacker installs an application—often legitimate—that serves as an intermediary for loading and executing the malicious DLLs. This process disguises malicious actions under the guise of legitimate application

⁶ Hijack Execution Flow: DLL Side-Loading T1574/002
<https://attack.mitre.org/techniques/T1574/002/>

⁷ Hijack Execution Flow: DLL Search Order Hijacking T1574/001
<https://attack.mitre.org/techniques/T1574/001/>

activity. This technique is particularly effective against traditional security measures and often succeeds in bypassing them.

Furthermore, when the malicious activity is designed to mimic the typical behavior patterns of a legitimate application, anomaly detection systems are less likely to flag it. This allows the malicious operations to proceed undetected, evading security mechanisms and operating under the radar without triggering alerts.

3. **Persistence:** To ensure the malicious code is reloaded upon each startup, in addition to employing COM Hijacking (T1546/0153⁸), the group utilizes custom services and modifications to the Registry (e.g., T1547/0014⁹).
4. **Use of Legitimate Platforms and Websites** (T11025): For example, the group leverages legitimate platforms such as Rentry.co, which allows for content sharing and redirection to various URLs, including Command and Control (C2) servers.

⁸ Event Triggered Execution: Component Object Model Hijacking T1546/015
<https://attack.mitre.org/techniques/T1546/015/>

⁹ Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder
<https://attack.mitre.org/techniques/T1547/001>

Tradecraft

In 2024, the National Cyber Directorate identified several unique tools developed by the group. Below is a detailed overview of some of these tools detected in the cyber domain:

Bugsleep (PE64,C++)

The backdoor enables attackers to execute commands remotely on the compromised system and establish a communication channel between the infected machine and the C2 server. Once access is established, the backdoor uses a scheduled task triggered every 43 minutes (customizable as needed) to maintain continuous communication. The malware connects to a hardcoded IP address and transmits sensitive information about the compromised system, including the computer name and the victim's username. This data undergoes obfuscation to conceal its true nature and evade detection by security systems.

Key Functions of the Backdoor:

1. **Shellcode Injection:**

Injects malicious code into various legitimate processes on the victim's system to mask its activity.

2. **File Exfiltration:**

Transfers files from the victim's system to the C2 server, facilitating data collection and analysis.

3. **Remote Command Execution:**

Allows the attacker to execute commands on the victim's endpoint for further malicious actions within the compromised system.

4. **Sleep Timer Updates:**

Modifies and configures the time intervals between communications (beacons) with the C2 server to enhance the tool's resilience against cybersecurity monitoring and detection.

Blackout (PE64, C++)

The backdoor operates as a Remote Administration Tool (RAT) and communicates with the C2 server using the HTTP protocol. Communication occurs through unique URLs and GET requests, which facilitate the transmission of various commands between the C2 server and the compromised system. Below are examples of such requests:

1. **Sending Commands to the Target:**

GET /questions

Used to transmit commands from the C2 server to the compromised system.

2. **Submitting Execution Results to the C2 Server:**

POST /about-us

Used to send the results of executed commands back to the C2 server.

3. **Uploading Files to the C2 Server:**

POST /awards/<filename>

Transfers files from the victim's system to the C2 server for analysis or further use.

4. **Downloading Files from the C2 Server:**

GET /support/<filename>

Retrieves additional files, such as updated payloads or configuration scripts, from the C2 server.

The tool collects and transmits basic information about the operating system, directories, files, and user accounts to the C2 server.

After the initial communication with the C2 server, the tool waits for further commands.

Once the tool receives a command from the C2 server, the output is sent back to the server via the URI /about-us, which displays the command execution results to the operator. Following this, a "sleep" command is issued for 20 seconds using the POST method before the tool requests a new command from the C2 server through /questions.

Main functions:

1. File Upload (dl):

This command uploads files from the compromised computer to the attacker's server. The upload is executed via a POST request to the following URI:

/awards/.

2. The "dl" command enables the downloading of files from the attacker's server. During the research, a DLL file named Revoke.dll was observed being downloaded from the server.

3. Loading and Execution of DLL Files ("Load"):

The Blackout tool leverages the Windows API function LoadLibrary to dynamically load the Revoke.dll file into a system process. Once the file is loaded, the tool uses the GetProcAddress function to locate the memory address of the export function named Run defined within this library.

The tool then invokes the Run function to execute the malicious code embedded in the DLL, enabling the execution of hidden code within the system environment. This method allows the tool to conceal its activity and evade detection by security systems that monitor standard API calls.

4. The Revoke.dll file, written in Go, functions as a backdoor that provides unauthorized access to the system. It uses the Windows LoadLibrary function to load the export function Run and execute the hidden code within the DLL. Additionally, the file employs the kbtls library, which implements a Key-Based TLS protocol to encrypt communication with the remote server. During the investigation, the following key was discovered within the file:
ZQpsae1yhmQjf3ofdHPYs/kmh1mEQaNaJzONUMfvrk0

During the TLS handshake process, the client (in this case, the victim) sends the Server Name Indication (SNI)¹⁰ to the server (controlled by the attacker) as part of the initial connection. The purpose of the SNI is to indicate to the server the specific domain the client intends to connect to, allowing the server to present the appropriate SSL certificate for the requested domain.

The SNI is sent within the "Client Hello" message, the first step in the handshake process. This mechanism enables the server to present the correct certificate, particularly in scenarios where multiple domains are hosted on the same IP address and use HTTPS.

It's important to note that the SNI is transmitted in plaintext, meaning third parties can observe the requested domain. However, the actual communication

¹⁰ Server Name Indication (SNI)
Request for Comments (RFC): 3546
<https://datatracker.ietf.org/doc/html/rfc3546>

between the client and server is encrypted, ensuring the content exchanged remains protected.

During the investigation, the tool communicated with the domain *magicallyday[.]com:443*, a domain known to have been used by the attacker at the time of the research.

This irregular SNI indicates an attempt to obscure or differentiate the communication, further complicating detection by traditional monitoring systems.

Further Analysis

It has not yet been definitively determined whether the value in the DLL file is static or dynamic. However, the current assessment suggests that the value remains constant and does not change.

MITRE ATT&CK TTPs:

Exfiltration Over C2 Channel (T1041)

File and Directory Discovery (T1083)

System Information Discovery (T1082)

System Owner/User Discovery (T1033)

Application Layer Protocol (T1071)

Ingress Tool Transfer (T1105)

IOCs:

97e844797181cd163b794529447219d3

15022bda37f65f33cad2bb5bf84a3900

283c87eaf43a3099217a6d6f01d9c4f3

963b75b2a747eec611265a22582d38e2

c3b990474c06086db4311c1553570174

6e5451b250731fcb5713bd043f406bf1

1drv[.]business

magicallyday[.]com

212.232.225[.]5

AnchorRat

A RAT that communicates with a C2 server and has the capabilities to upload, download, and load various DLL files, depending on the command sent by the operator.

Downloading and Loading DLLs via COM Hijacking:

COM Hijacking is a common attack technique where attackers exploit the Component Object Model (COM) mechanisms in Windows operating systems to gain control over the system, execute malicious code, and perform unauthorized actions. COM is a legitimate mechanism that allows programs to communicate with each other, but attackers can exploit the COM registry structure to replace legitimate objects with malicious ones, thus hiding their activity.

By modifying registry entries, attackers can configure COM objects to point to malicious files instead of the original ones. When legitimate software tries to access a COM object, it might execute the malicious code instead of the intended code. This allows the attacker to gain control over the system and perform unauthorized actions.

The first command executed by *AnchorRAT* is downloading a DLL. To establish persistence, the RAT uses the COM Hijacking technique by modifying a registry key:

```
HKCU\Software\Classes\CLSID\{0358B920-0AC7-461F-98F4-58E32CD89148}\InProcServer32.
```

The second command downloads and registers the RAT's executable file (AnchorRAT) as a service named OneDriveStandaloneUpdaterService, creating

the false appearance of a legitimate Microsoft service and providing an additional means of persistence.

The malware uses the HTTPS protocol (port 443) and transmits data in JSON format, enabling relatively covert communication with the C2 server.

Commands supported when using HTTPS:

Description	Method	Port	URL
Used for "registration" (signup) with the C2 server. During registration, details such as FQDN, hardware ID (hardwareId), whether the RAT is running as an administrator (admin), and the sleep time value are sent.	POST	443	https://ulpanim[.]wiki/signup
Used to receive new commands from the C2 server.	GET	443	https://ulpanim[.]wiki/order
Sends the results of executed commands back to the C2 server.	POST	443	https://ulpanim[.]wiki/order
Used to receive a process name and execution instruction from the C2 server.	GET	443	https://ulpanim[.]wiki/contact
Sends the process name and execution instruction or additional updates to the C2 server.	POST	443	https://ulpanim[.]wiki/contact
Used to download a file from the C2 server to the compromised system.	GET	443	https://ulpanim[.]wiki/shop
Initial call to the victim's address to collect information and prepare for file upload later.	GET	443	https://ulpanim[.]wiki/detail

Description	Method	Port	URL
After collecting the information, uploads the file to the C2 server as a follow-up to the previous GET request.	POST	443	https://ulpanim[.]wiki/detail
Used to download a DLL file for persistence using COM injection.	GET	12345	https://ulpanim[.]wiki/support1
Used to download a PE file for persistence using a "service."	GET	678910	https://ulpanim[.]wiki/support2
Used to download an additional file or component for another persistence method or added functionality.	GET	111213	https://ulpanim[.]wiki/support3

MITRE ATT&CK TTPs:

Event Triggered Execution: Component Object Model Hijacking (T1546.015)

Create or Modify System Process: Windows Service (T1543.003)

Ingress Tool Transfer (T1105)

Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder (T1547.001)

IOCs:

ac819c8e223c20df9bb9a80ab5c20e4a

ulpanim[.]wiki

CannonRat

A remote-control tool that communicates with a C2 server to receive commands and exchange information between the victim's computer and the server. In the analysis of this version of the tool, it was found that it allows the attacker to run various types of files, including DLL files, which may be used to execute additional malicious code, install additional software components, or perform further malicious activities on the victim's system.

Some of the commands supported by the RAT:

Sleep, pwd, cd, upload, download, runexe, rundll

Below is an overview of the main steps of the RAT's operation:

1. The RAT creates a folder named WinSys and copies itself to the path:
%localappdata%\WinSys\Secure System.exe .
2. The tool writes a DLL file used as a loader to the following path:
%localappdata%\WinSys\coreglobconfig.dll\ .

This file is intended to load and execute additional malicious code on the victim's computer, thereby enabling the operator to expand access and control.

3. The RAT modifies the registry key:
HKCU\Software\Classes\CLSID\{10D62541-90D0-42FE 848C-0DBC1AC42EDA}\InProcServer32
to point to the DLL file it created (coreglobconfig.dll).

The InProcServer32 key under the CLSID:

{10D62541-90D0-42FE 848C-0DBC1AC42EDA}

is used to establish persistence via COM hijacking.

When a specific path to a DLL is set under InProcServer32 in the registry for a given CLSID, any access to this COM object or any action in Windows that loads this CLSID will automatically load the specified DLL (coreglobconfig.dll). This technique exploits Windows' DLL loading mechanism to execute malicious code. The loaded DLL performs the required actions (such as executing malicious operations) without raising suspicion.

4. The same DLL file (coreglobconfig.dll) re-executes the CannonRAT (Secure System.exe) from its new installation path: %localappdata%\WinSys\Secure System.exe

MITRE ATT&CK TTPs:

Ingress Tool Transfer (T1105)

Create or Modify System Process: Windows Service (T1543.003)

Event Triggered Execution: Component Object Model Hijacking (T1546.015)

IOCs:

b15dd02164a9bb53356ba1d748301bf7

50da52b517c708a0d409ba20fd00b10f

7d5b1ae57599940faf51e0b38f4824bd

f700378578df895e80c0dfeea68fe694

spaziogroup[.]org

Neshta

Neshta is an old malware variant that continues to be prevalent in the cyber domain. It was first identified in 2003 and has previously been linked to the BlackPOS malware. Neshta injects malicious code into infected files. This malware often infiltrates organizational networks via unintended user downloads or through other malware.

Neshta primarily targets Windows operating system files and can facilitate attacks on network shares and removable storage devices (e.g., USB drives). Over the years, the malware has undergone upgrades and modifications, leading to a relatively large number of its variations.

As part of MuddyWater's use of legitimate tools with dual-use capabilities (both legitimate and malicious), such as Venom and SimpleHelp, Neshta has been observed as part of their operations. This integration allows the malware to spread and infect additional files, leveraging legitimate tools as a disguise for malicious activities. It aids in gaining control over the system or access to it without raising suspicion.

Investigation Findings: re.exe

During an investigation held by the INCD, a file named re.exe was discovered, which utilizes the legitimate SimpleHelp tool in the following manner:

- **Downloader/Dropper Functionality:**
re.exe is a downloader/dropper communicating with a C2 server to download a JWrapper installation package containing the SimpleHelp remote control software.

- File Infection Mechanism:

The malware infects all executable files that are not system files by adding malicious code to the beginning of the files while preserving their normal functionality. This ensures the malware spreads covertly within the system without disrupting the standard operation of files and system applications.

- Registry Modification:

The malware modifies the registry key:

HKCR\exefile\shell\open\command\

This change causes the malicious file svchost.com to execute before the original file is launched.

Path and Execution of svchost.com:

The malicious file svchost.com is written to C:\Windows\ - It is used to run the malware via the registry, executing the malicious code before launching the original file requested by the user.

- Temporary File Creation:

When an infected file is executed, a clean copy of the file is written to a temporary folder: C:\Users\<username>\AppData\Local\Temp\3582-490\

The svchost.com file then runs the clean copy of the file to ensure the original software continues to function correctly. This behavior minimizes suspicion from the user while maintaining the malware's operation.

MITRE ATT&CK TTPs:

T1554 – Compromise Host Software Binary

T1574 - Hijack Execution Flow

T1105 – Ingress Tool Transfer

T1219 – Remote Access Software

IOCs:

297f77096dfb485641b4594c83b32a7c

c4406f5fff870955af772d676a30a0cf

25B6F3F4B13BC53DD4981915CDD95E33

2F257EAD7F42DF4E9115DDAB552E77E4

SOCKS5

During the research, several tools utilizing the SOCKS5 protocol were identified. This protocol establishes a secure connection between the compromised system and the attacker's server via a proxy connection. SOCKS5 helps bypass security mechanisms, firewalls, and NAT, effectively hiding the connection's origin and maintaining anonymity. Furthermore, the protocol allows data to be transmitted covertly and efficiently, facilitating prolonged and effective attacks while evading detection or blocking by security systems.

As part of the research, the attacker's use of the protocol was analyzed, leading to the discovery of the tools `service.exe` and `ego.exe`, which exploit the protocol and open-source libraries to create encrypted and secure connections. The connection is established using an IP address and port provided as command-line arguments. `service.exe` utilizes the open-source library "Gomux" after establishing the connection, while `ego.exe` uses the "yamux" library. Both libraries enable multiplexing, allowing the management of multiple bidirectional streams over a single connection.

Details on ego.exe

The `ego.exe` tool is written in Go and packed with the UPX Packer to obfuscate its purpose. It employs additional obfuscation techniques, such as removing function names from the binary, making it difficult to analyze and understand the executed functions. The tool also includes anti-debugging mechanisms to hinder analysis and prevent detection of its malicious behavior during testing.

Additionally, the tool encrypts and decrypts data using the AES encryption algorithm, ensuring the transmitted information is concealed and the

communication is protected. Upon execution, ego.exe displays a fake error message to reduce user suspicion and mask its true activity. Immediately afterward, the tool begins communicating with the attacker's IP address over TCP port 443, enabling it to bypass firewalls and avoid detection of its communication.

Details on service.exe

The service.exe tool operates in a manner very similar to ego.exe, with the primary difference being its use of the Gomuxlibrary and the encryption library golang.org/x/crypto.

github.com/things-go/go-socks5

github.com/hashicorp/yamux

github.com/Acebond/gomux

golang.org/x/crypto

MITRE ATT&CK TTPs:

T1497.001 - Virtualization/Sandbox Evasion: System Checks

T1140 - Defense Evasion: Deobfuscate/Decode Files or Information

T1509.003 - Command and Scripting Interpreter - Windows Command Shell

User Execution - T1204

T1072 - Software Deployment Tools

T1573 - Encrypted Channel

T1090 – Proxy

IOCs:

c851e849c8442727eac69225203ee7f7

d3e259a8caa7e23e89453a387caa3a15

C1EF5F29C1811444E1E96C25E667F18D

fd5b55c1b97bef7b4f3114d39984e597

9077295f0eb9db45fc495a04637ee197

C28F276677F7E85E043E0DBEB5065255

C2 Frameworks

As part of research conducted by the National Cyber Directorate, it was found that some of the tools presented in this report interacted with server-side C2 frameworks used by the attacker as core infrastructure for operating malicious tools. During the research, two new attack frameworks used by the attacker were identified: Sad C2 and Havoc C2.

Sad C2

The framework supports the *Pheonix* and *BlackPearl* malware, as detailed later in the report. The server-side component of the framework is written in Python and provides the attacker with remote management and control capabilities over the compromised systems, fully integrating with the malware. According to initial analysis, the framework appears to have been independently developed by the attacker, as no variations or similar sources were found in public channels.

The framework utilizes FastAPI for communication, where the client initiates interaction with the server. Subsequently, API requests are sent to perform various actions, such as data retrieval, information updates, and execution of additional functions based on the attacker's requirements.

The attacker, via the C2 server, communicates with the client side by sending and receiving commands, as well as managing file transfers (uploads and downloads). Communication between the client and the server under the attacker's control is secured using symmetric AES encryption with the key STAGER_AES_KEY. This encryption ensures that data transferred between the compromised system and the attacker's server remains private, preventing detection and neutralization by security systems.

The framework supports multiple communication mechanisms, including DNS and HTTPS protocols, allowing the attacker to communicate with compromised systems flexibly and covertly. As part of the framework's operation, the following components were identified on the C2 server:

Components Identified on the C2 Server

- Command History File - "archer/history" - This file contains a record of commands sent by the attacker.
- Global Files for File Management and Imports: Global.py
Global.cpython-312.pyc
- DNS and HTTPS Handlers:
Listener/DNS.py
Listener/HTTTPs.py
http_tools.py.

```
def modify_client_exe(new_ip, new_port):
    port_to_search = (55321).to_bytes(2, byteorder="little")
    new_data_cc, new_data_stager = b"", b""
    encoded_ip = encode_bytes("999.999.999.9999".encode(), sum_key)
    stager_encoded_ip = encode_bytes("999.999.999.9999".encode(), satger_sum_key)
    try:
        with open("packages/clean client/CC_HTTP_NA.exe", 'rb') as f:
            with open("packages/clean client/StagerForC2.exe", 'rb') as g:
                data_cc = f.read()
                data_stager = g.read()
```

Sad C2 Server Side Script

```
def modify_client_exe(new_ip, new_port):
    port_to_search = (55321).to_bytes(2, byteorder="little")
    new_data_cc, new_data_stager = b"", b""
    encoded_ip = encode_bytes("999.999.999.9999".encode(), sum_key)
    stager_encoded_ip = encode_bytes("999.999.999.9999".encode(), satger_sum_key)
    try:
        with open("packages/clean client/CC_HTTP_NA.exe", 'rb') as f:
            with open("packages/clean client/StagerForC2.exe", 'rb') as g:
                data_cc = f.read()
                data_stager = g.read()
```

SAD C2 FRAMEWORK – SERVER SIDE

```
from uvicorn import run as runHttpServer
from DB.Service import (
    ask_shell,
    upload_file,
    shell_it,
    wake_agent_up,
    sleep_agent,
    download_file,
    remove_agent,
    kill_yr_self,
    set_communication,
    Cacher, shell_hit, search_victim
)
from Listener.DNS import run_dns_server
from Listener.HTTPs import app
from helpers.print_helper import print, prompt, _setUpCommandLineSeggestion
from DB.DataBase import Victim, Command, ShellRCCommand
from Global import *
from threading import Thread, ThreadError
from time import sleep

def http_bind(host="0.0.0.0", port=8082, debug=False):
    runHttpServer(app, host=host, port=port, log_level= None if debug else "critical")
def dns_bind(host="0.0.0.0", port=53, protocol="udp"):
    run_dns_server(host, port, protocol)
# html_Completer = WordCompleter(['test', 'target', 'tartan', 'tool'])

# text = prompt('Enter HTML: ', completer=html_completer)
# print('You said: %s' % text)
# 157.20.182.102
```

BlackPearl's Sad C2 Server Side Py Script

```
case 'help' | 'h' | 'man':
    print(['', "\n"])
    print(['', "-----"])
    print(['', "help", "h", "man ->"])
    print(['', "sleep", "sleep ->"])
    print(['', "wake", "wake up ->"])
    print(['', "upload file", "file upload", "up", "upl", "file up ->"])
    print(['', "download file", "file download", "download", "down", "file dn ->"])
    print(['', "list session", "get session list", "session list", "session ls", "ls ->"])
    print(['', "ls term", "ls term ->"])
    print(['', "attach shell", "attach shell ->"])
    print(['', "kill", "agent kill", "kill agent", "kill your self", "self kill ->"])
    print(['', "shellrc", "shell", "shell ->"])
    print(['', "select session", "session select", "choose session", "get session", "pawn ->"])
    print(['', "-----"])
    print(['', "\n"])
```

BlackPearl's Sad C2 Server Side Py Script

SAD C2 Tools

TreasureBox (PE64, C++):

Loader Tool for Deploying the "BlackPearl" RAT

The Loader (TreasureBox) utilizes a hardcoded key, "12345", to decrypt and extract the payload using Huffman-encoded XPRESS compression. The payload itself is stored in an additional encoded format, which is decrypted using a key named "LOGIMAN".

After decryption, the BlackPearl payload is injected into the runtimebroker.exe process using standard Windows API functions, such as:

- OpenProcess
- VirtualAllocEx
- WriteProcessMemory

Once the injection process is complete, the payload is executed via the CreateRemoteThread function.

IOCS:

044365681b0e781292e79c19906f379b7c3e0d5a404b19b56ed7b447b75d1485

BlackPearl (PE64, C++)

A remote access tool that provides the following functionalities:

- Full control over the compromised system.
- File transfer (upload and download).
- Creation and management of shells.
- Switching between communication protocols (HTTP and DNS).
- Utilization of sleep timers to evade detection by security measures.

The payload creates a new Screensaver file named Bubbles.scr, which is essentially a copy of the TreasureBox loader, located at: C:\ProgramData\arch.1.65.com.microsoft\Bubbles.scr.

The specified path is passed to the Payload (Bubbles.scr) via a Pipe named Archer, which serves as an inter-process communication channel.

Subsequently, the Loader modifies the Registry under the path: HKEY_CURRENT_USER\Control Panel\Desktop, setting a reference to the newly created file (Bubbles.scr). This ensures that the Loader is re-executed after a system reboot.

Additionally, the RAT employs a synchronization/mutex object named Buta, which ensures that only one instance of the RAT is running on the system at any given time.

C2 Communication:

The initial beacon is sent via an HTTP POST request, containing victim identification data in the following format:

<username>|<computer_name>|<random_number>

The tool features an encryption mechanism, ensuring that data transmitted to the C2 server is encrypted using AES-128 in CBC mode. With specific Flags set, the RAT switches to DNS-based communication instead of HTTP

Description	Command
Download a file from the server using HTTP or DNS.	DOWN
Terminate the RAT process.	KILL
Disable the Time Flag to execute random actions instead of waiting (Sleep).	TIME
Enable the Time Flag, leading to more complex operations along with sleep timers.	TIMT
Write a command to the Shell via a POST request to large_number_as_string>>/sh/.	WT
Create a Host ID, send a Beacon, and open a Shell.	0x404
Switch communication mode between HTTP and DNS.	CP
Create a new remote Shell.	RC
Terminate a remote Shell based on a unique identifier.	RM
Upload a file to the server.	UP

```

case 'help' | 'h' | 'man':
    print(['', '\n'])
    print(['', "-----"])
    print(['', "help", "h", "man ->"])
    print(['', "sleep", "sleep ->"])
    print(['', "wake", "wake up ->"])
    print(['', "upload file", "file upload", "up", "upl", "file up ->"])
    print(['', "download file", "file download", "download", "down", "file dn ->"])
    print(['', "list session", "get session list", "session list", "session ls", "ls ->"])
    print(['', "ls term", "ls term ->"])
    print(['', "attach shell", "attach shell ->"])
    print(['', "kill", "agent kill", "kill agent", "kill your self", "self kill ->"])
    print(['', "shellrc", "shell", "shell ->"])
    print(['', "select session", "session select", "choose session", "get session", "pawn ->"])
    print(['', "-----"])
    print(['', '\n'])

```

BLACKPEARL – SERVER SIDE PY

```

from uvicorn import run as runHttpServer

from DB.Service import (
    ask_shell,
    upload_file,
    shell_it,
    wake_agent_up,
    sleep_agent,
    download_file,
    remove_agent,
    kill_yr_self,
    set_communication,
    Cacher, shell_hit, search_victim
)

from Listener.DNS import run_dns_server
from Listener.HTTPs import app
from helpers.print_helper import print, prompt, _setUpCommandLineSeggestion
from DB.DataBase import Victim, Command, ShellRcCommand
from Global import *
from threading import Thread, ThreadError
from time import sleep

def http_bind(host="0.0.0.0", port=8082, debug=False):
    runHttpServer(app, host=host, port=port, log_level= None if debug else "critical")

def dns_bind(host="0.0.0.0", port=53, protocol="udp"):
    run_dns_server(host, port, protocol)
    # html_completer = WordCompleter(['test', 'target', 'tartan', 'tool'])

    # text = prompt('Enter HTML: ', completer=html_completer)
    # print('You said: %s' % text)
    # 157.20.182.102

```

BLACKPEARL – SERVER SIDE PY

Summary

TreasureBox and BlackPearl facilitate communication with the C2 server through both DNS and HTTP listeners, allowing flexibility and full control over compromised infrastructure. Additionally, the server logs all executed commands, enabling the attacker to track activities and obtain information about the infected system. Communication is managed via scripts such as DNS.py and HTTPS.py, with main.py serving as the primary orchestrator for session management and system control.

Pheonix (C++, PE64)

A binary masquerading as a Word file, designed to download payloads from the C2 server.

- Dynamically resolves API functions and performs checks to detect analysis or monitoring environments, such as tcpview.exe, ida.exe, and others.
- If an analysis environment (e.g., sandbox) is detected, it may halt execution or modify its behavior.
- Attempts to execute the following process:
%localappdata%\Microsoft\Internet Explorer\IEMonitor\iemonitor.exe
- If successful, it establishes an encrypted AES communication channel and sends the encryption key to the C2 server via an HTTP POST request.
- The C2 server is expected to respond with an encrypted payload (likely shellcode or an executable), which is injected into iemonitor.exe.
- Though no additional payloads were identified during analysis, it is configured to execute arbitrary code as required.

IOCS:

6f6869fe0d47ef2abbd30651f6348de3868e4e2f642e30f468bc8376ec30b150

CC_HTTP_NA.exe (PE64, C++)

A backdoor allowing remote control over the victim's machine, file system manipulations, command execution, and payload deployment.

Strings are decrypted using a subtraction cipher with a static 32-byte key. Uses fake IP addresses (e.g., 999.999.999.9999), indicating it may be a development version. Performs checks for virtual environments via Registry queries, e.g., `HARDWARE\ACPI\DSMT\VBOX__`.

It copies itself to `%localappdata%\MicrosoftEdgeUser`

`DataSafetyTipssafetytips.exe` and additional DLL files (`provider270.dll` and `provider360.dll`) during runtime based on specific conditions. It communicates with the C2 server via HTTP, secured with AES encryption.

Sends victim details (username, computer name, architecture, etc.) to the `bingo/` path. Supports functions such as executing `cmd.exe` commands, file creation, exfiltration, and managing job objects via pipes.

IOCS:

153128c13808b275b6f00bda3a616ee6fbb26f21d9124b13ab6daf1c7e7ff48e

HAVOC Framework

In contrast to Sad C2, which operates as a "closed" command and control (C2) framework, this HAVOC ¹¹Framework is publicly available. It supports both HTTP/HTTPS and SMB protocols, facilitating the creation of executables, shellcode, and DLLs. This versatility enables MuddyWater to conduct a broad range of attacks, including the injection of malicious code into existing processes and the automation of file execution within targeted environments.

The group routinely employs various publicly available tools, such as Mythic C2, Chisel, Venom, and GO-SOCKS5. These tools have been observed in use over the past year, alongside custom-developed tools, as detailed in the report

¹¹ HAVOC

<https://github.com/HavocFramework/Havoc>

Summary

Summary of Findings and Investigation Report

MuddyWater has been active since 2017, and has undergone significant evolution since its inception. Initially, the group focused on conducting targeted cyberattacks in the Middle East. Over time, it expanded its operations and became a significant player in both the Israeli and global cyber arenas.

One of MuddyWater's most prominent characteristics is the continuous development of its offensive toolkit. The group transitioned from using generic tools to developing custom malware tailored specifically to its objectives. Additionally, the group invests substantial resources in developing new methods to evade detection systems, enhancing its efficiency and complicating efforts to identify its activities.

While MuddyWater's operational focus remains centered in the Middle East, the group is also active in other countries, including Israel, Turkey, Pakistan, the United Kingdom, and the United States. In the past year, there has been a notable increase in the group's activity in Israel, particularly following significant regional events, such as the "Iron Swords" War. This increase highlights the group's ability to exploit geopolitical events to expand its operations and achieve the objectives of Iran's supreme leadership.

Preferred Targets

The group focuses on long-term strategic intelligence gathering, with an emphasis on infiltrating critical sectors, including:

- **Aviation**
- **Healthcare**
- **Telecommunications**

- IT
- SMBs

The group selects targets that hold large amounts of sensitive information, such as patient data and critical infrastructure systems, as well as systems with known vulnerabilities. This selection aims to gain access to strategically valuable information and cause significant harm to organizations. Additionally, SMBs are often easier targets due to their limited resources.

The attacks are primarily conducted in a **CNE** (Computer Network Exploitation) framework using:

- Advanced social engineering techniques.
- Remote Management Tools (RMM).
- Exploitation of known vulnerabilities to gain access and establish persistence within organizational networks.

Techniques and Key Tools

Spear-Phishing and Wide-Scale Phishing

- Sending malicious emails containing links or files disguised as legitimate information.
- Utilizing spoofing or legitimate email accounts to exploit existing email threads for injecting malicious code.

Technical Highlights

- Communication Protocols: HTTP and DNS
 - HTTP: The group leverages HTTP to transfer commands and data in traffic that appears legitimate, making it difficult for traditional security systems to detect.

- DNS: Used as a covert channel for data transfer, exploiting the protocol's nature and the fact that many organizations do not routinely monitor DNS traffic.
- Data Encryption
 - Use of algorithms such as AES-128 to maintain anonymity and secure communication.
 - Flexible Communication Management
 - Deployment of dedicated listeners on the C2 server alongside scripts, enabling the operator to dynamically and flexibly manage operational configuration settings.

Current Trends and Near-Future Outlook

In 2024, MuddyWater expanded its toolkit, placing considerable emphasis on the development of custom, tailored malware. This shift reflects the group's ongoing efforts to enhance its operational capabilities and increase the effectiveness of its attacks across diverse environments.

Based on an analysis of past trends, it is expected that the group will continue to evolve both technologically and operationally, adopting innovative strategies to achieve its objectives. This evolution may include an intensification of its activities within the Israeli cyber domain, as well as advancements in its infiltration techniques and persistence within targeted computer systems.

***** End of Document *****